FunnyFileManager

Архитектурная документация

Терминальный двухпанельный файловый менеджер

Вдохновлён Far Manager

Написан на Rust с использованием TUI-фреймворка Ratatui

Оглавление

1 Введение 4
1.1 Ключевые возможности 4
1.2 Технологический стек 4
2 Структура проекта 5
2.1 Организация файлов 5
2.2 Диаграмма зависимостей модулей 5
3 Основные структуры данных 6
3.1 DirEntry 6
3.2 SortMode 6
3.3 PanelSide 6
3.4 Panel 7
3.4.1 Методы Panel 7
3.5 Диаграмма состояния панели 7
3.6 Viewer 8
3.7 App (Состояние приложения) 8
3.8 Диаграмма состояния приложения 9
4 Компоновка пользовательского интерфейса 10
4.1 Структура экрана 10
4.2 Размеры элементов 10
4.3 Цветовая схема 11
4.4 Функции отрисовки UI 11
5 Цикл событий и обработка клавиатуры 12
5.1 Главный цикл событий 12
5.2 Горячие клавиши 13
5.2.1 Клавиши навигации 13
5.2.2 Функциональные клавиши 13
5.2.3 Клавиши отметки 13
5.2.4 Быстрый поиск 13
5.3 Приоритет оверлеев 15
6 Обработчики файловых операций 16
6.1 Обзор обработчиков 16
6.2 Поток операции копирования 16
6.3 Поток операции перемещения 17
6.4 Команды командной строки 17
7 Вспомогательные функции 19
7.1 Обзор utils.rs 19
7.2 Логика format_size() 19
7.3 Алгоритм unique_destination() 19
7.4 Рекурсивный алгоритм copy_dir_all() 20
8 Интеграция с редактором 21
8.1 Стратегия выбора редактора 21
8.2 Последовательность handle_edit() 22
9 Паттерны потока данных 23
9.1 Поток навигации по панели 23
9.2 Поток быстрого поиска 24
10 Обработка ошибок 25

10.1 Стратегия обработки ошибок 25
10.2 Поток обработки ошибок 25
11 Архитектура тестирования 26
11.1 Уровни тестирования 26
11.2 Инфраструктура тестирования 26
11.3 Точки мутационного тестирования 26
12 Архитектурные решения 27
12.1 Ключевые проектные решения 27
12.1.1 1. Двухпанельная архитектура 27
12.1.2 2. Паттерн стека оверлеев 27
12.1.3 3. Сохранение выделения 27
12.1.4 4. Переименование с запасным вариантом для перемещения 27
12.1.5 5. Уникальное именование файлов назначения 27
13 Руководство по расширению 28
13.1 Добавление новых возможностей 28
13.1.1 Новая команда F-клавиши 28
13.1.2 Новый оверлей 28
13.1.3 Новый режим сортировки 28
13.1.4 Новая команда 28
13.2 Точки конфигурации 28
14 Заключение 29
14.1 Архитектурные особенности 29

1 Введение

FunnyFileManager (FFM) — терминальный двухпанельный файловый менеджер, написанный на Rust. Он предоставляет пользовательский интерфейс в стиле Far Manager с клавиатурной навигацией и файловыми операциями.

1.1 Ключевые возможности

- **Двухпанельная компоновка** для эффективных файловых операций между каталогами
- **Цветовая схема Far Manager** (тёмно-синий фон, бирюзовый текст)
- **F-клавиши** для всех основных операций
- **Быстрый поиск** с комбинациями Alt+буква
- **Закладки** для часто посещаемых каталогов
- **Встроенный просмотрщик файлов** для текстовых файлов
- **Интеграция с внешним редактором** с приоритетом VS Code

1.2 Технологический стек

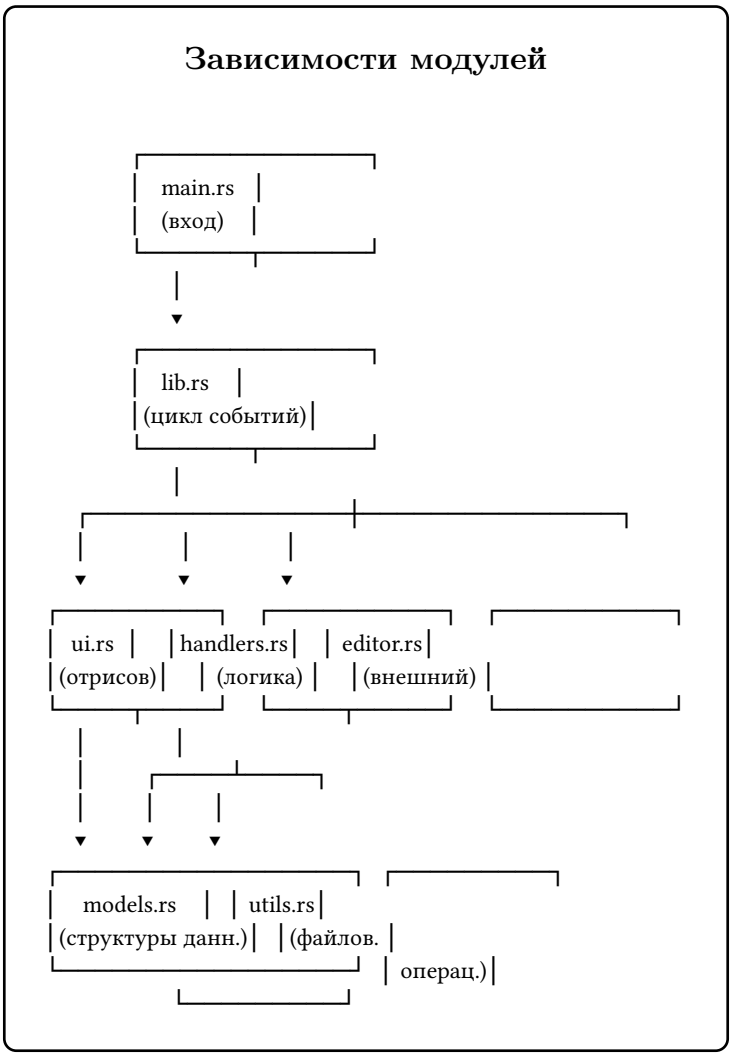
Компонент	**Технология**
Язык	Rust 2021 Edition
TUI-фреймворк	Ratatui
Терминальный бэкенд	Crossterm
Тестирование	Встроенное + Tarpaulin для покрытия

2 Структура проекта

2.1 Организация файлов

```
FunnyFileManager/
├── src/
│   ├── main.rs          # Точка входа (тонкая обёртка)
│   ├── lib.rs           # Публичный API, цикл событий
│   ├── models.rs        # Основные структуры данных
│   ├── ui.rs            # Отрисовка TUI
│   ├── handlers.rs      # Обработчики событий
│   ├── utils.rs         # Вспомогательные функции
│   └── editor.rs        # Интеграция с редактором
├── tests/
│   └── integration_test.rs
├── Cargo.toml
└── CLAUDE.md
```

2.2 Диаграмма зависимостей модулей



3 Основные структуры данных

3.1 DirEntry

Представляет одну запись файловой системы (файл или каталог).

```
pub struct DirEntry {
    pub name: String,    // Имя файла или ".."
    pub is_dir: bool,    // Флаг каталога
    pub size: u64,       // Размер файла в байтах
}
```

3.2 SortMode

Перечисление для сортировки содержимого каталога.

```
pub enum SortMode {
    Name,       // По алфавиту (без учёта регистра)
    Extension,  // По расширению, затем по имени
    Size,       // По размеру (по убыванию)
}
```

Режим	Описание	Метка
Name	Алфавитная сортировка без учёта регистра	“Name”
Extension	Группировка по расширению, затем по алфавиту	“Ext”
Size	Сначала большие файлы	“Size”

3.3 PanelSide

Простое перечисление, определяющее активную панель.

```
pub enum PanelSide {
    Left,   // Левая
    Right,  // Правая
}
```

3.4 Panel

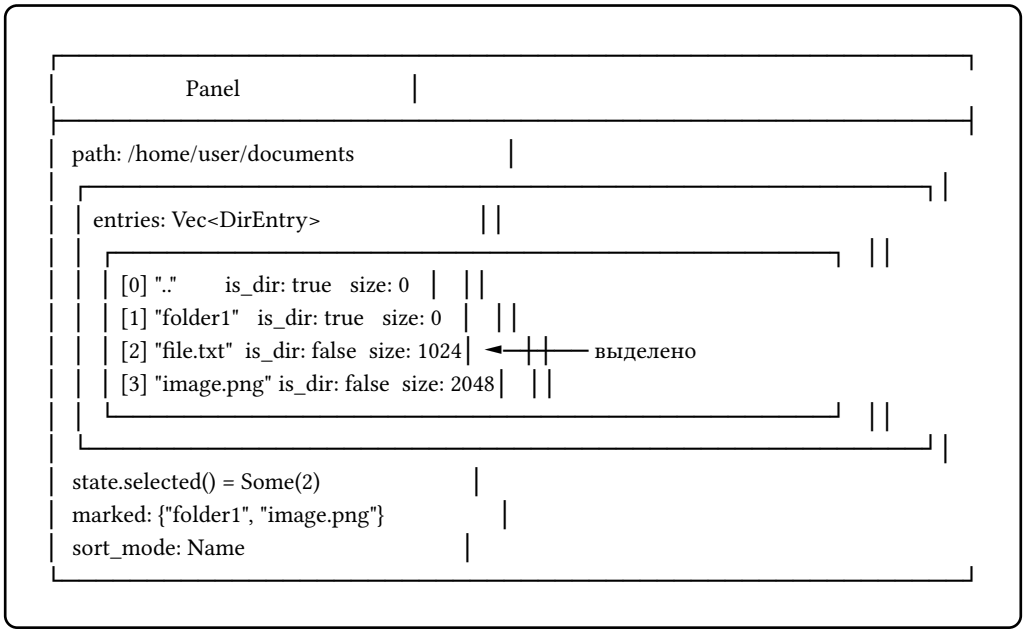
Структура Panel представляет одну половину двухпанельного интерфейса.

```rust
pub struct Panel {
    pub path: PathBuf,          // Текущий каталог
    pub entries: Vec<DirEntry>,  // Содержимое каталога
    pub state: ListState,        // Состояние выделения (ratatui)
    pub marked: BTreeSet<String>,// Отмеченные файлы
    pub sort_mode: SortMode,     // Текущая сортировка
}
```

3.4.1 Методы Panel

Метод	Назначение
new(path)	Инициализация панели для указанного пути
load_entries()	Обновление содержимого каталога
next() / previous()	Навигация с переходом по кругу
enter()	Вход в каталог или возврат
toggle_mark_selected()	Отметить/снять отметку для групповых операций
find_first_matching(prefix)	Быстрый поиск с начала
find_next_matching(prefix)	Быстрый поиск с циклическим переходом
selected_path()	Получить полный путь выделенного элемента

3.5 Диаграмма состояния панели



3.6 Viewer

Структура оверлея просмотрщика файлов (только чтение).

```rust
pub struct Viewer {
    pub title: String,      // Имя просматриваемого файла
    pub lines: Vec<String>, // Содержимое файла (построчно)
    pub scroll: u16,        // Текущая позиция прокрутки
}
```

3.7 App (Состояние приложения)

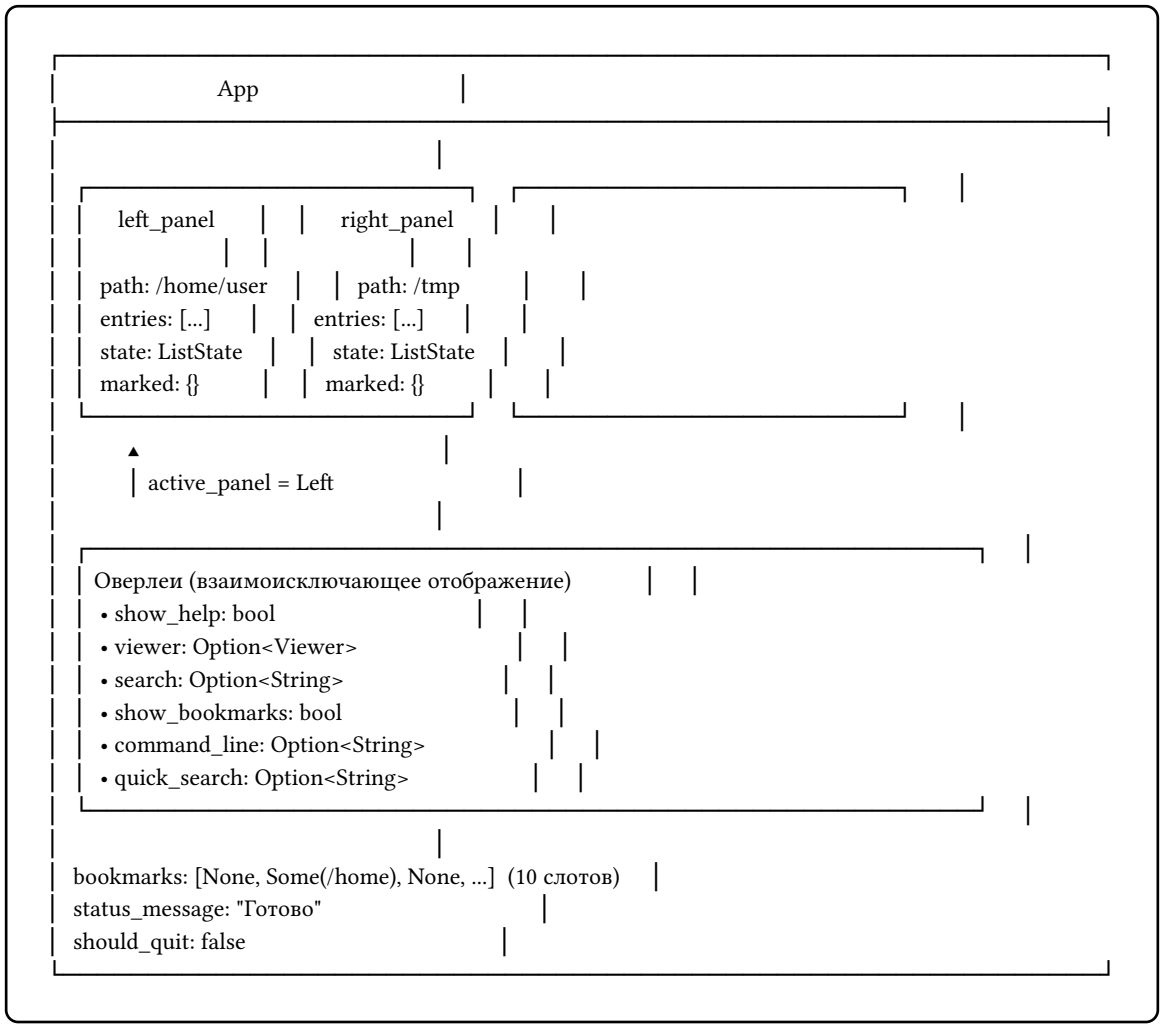
Главный контейнер состояния приложения, хранящий все данные времени выполнения.

```rust
pub struct App {
    // Панели
    pub left_panel: Panel,
    pub right_panel: Panel,
    pub active_panel: PanelSide,

    // Оверлеи
    pub show_help: bool,
    pub viewer: Option<Viewer>,
    pub search: Option<String>,
    pub command_line: Option<String>,
    pub show_bookmarks: bool,
    pub quick_search: Option<String>,

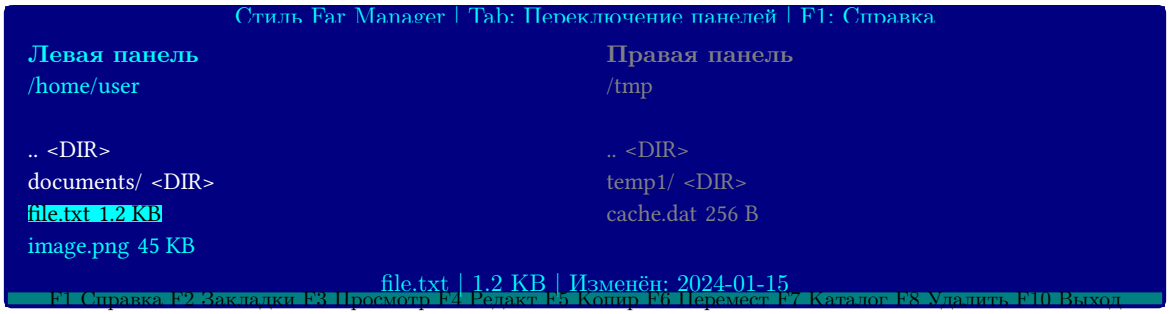
    // Состояние
    pub bookmarks: Vec<Option<PathBuf>>, // 10 слотов
    pub should_quit: bool,
    pub status_message: String,
}
```

3.8 Диаграмма состояния приложения



4 Компоновка пользовательского интерфейса

4.1 Структура экрана



4.2 Размеры элементов

Компонент	Высота	Описание
Заголовок	1 строка	Название приложения и подсказки
Панели	Остаток	Разделение 50/50 по горизонтали
Строка состояния	2 строки	Информация о файле и быстрый поиск
Панель команд	2 строки	Отображение F-клавиш

4.3 Цветовая схема

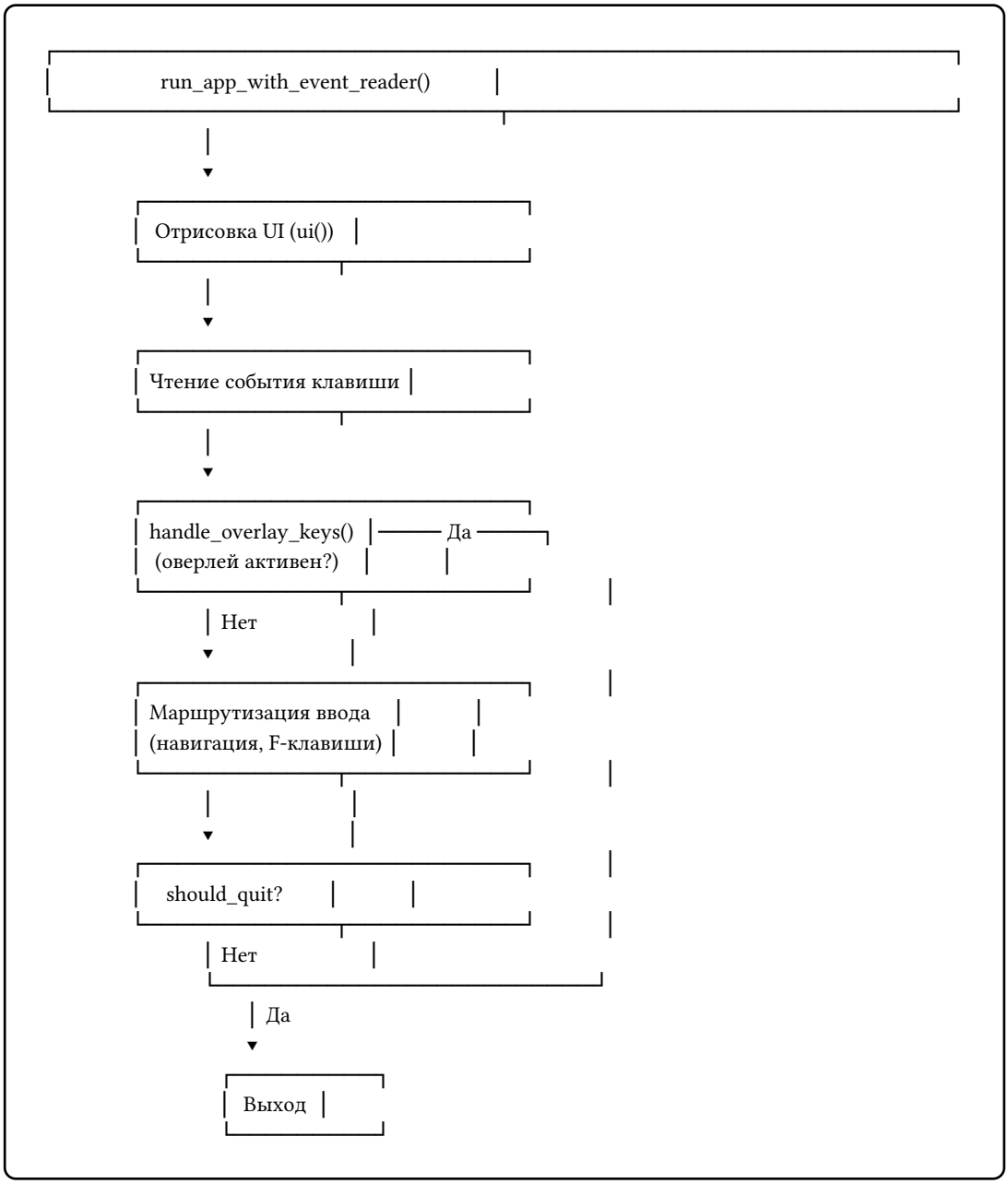
Элемент	Цвет	RGB-значение
Фон	Far Blue	RGB(0, 0, 128)
Активная рамка	Бирюзовый жирный	RGB(0, 255, 255)
Неактивная рамка	Тёмно-синий	RGB(0, 0, 139)
Каталоги	Белый жирный	RGB(255, 255, 255)
Файлы	Бирюзовый	RGB(0, 255, 255)
Размеры файлов	Светло-бирюзовый	RGB(224, 255, 255)
Выделение	Чёрный на бирюзовом	Фон: Бирюзовый, Текст: Чёрный
Отмеченные файлы	Жёлтый	RGB(255, 255, 0)

4.4 Функции отрисовки UI

Функция	Назначение
ui(frame, app)	Главный координатор отрисовки
render_panel(frame, panel, area, is_active)	Отдельная панель с записями
render_header(frame, area)	Верхний баннер
render_status(frame, app, area)	Информация о файле / быстрый поиск
render_command_bar(frame, area)	F-клавиши
render_overlays(frame, app)	Оверлеи справки, просмотра, поиска
centered_rect(pct_x, pct_y, area)	Расчёт размеров всплывающего окна

5 Цикл событий и обработка клавиатуры

5.1 Главный цикл событий



5.2 Горячие клавиши

5.2.1 Клавиши навигации

Клавиша	Действие
↑ / ↓	Перемещение выделения вверх/вниз (с переходом по кругу)
← / →	Переключение между панелями
Tab	Переключение между панелями
Enter	Вход в каталог или возврат (..)
Home	Переход к первому элементу
End	Переход к последнему элементу
PgUp / PgDn	Постраничная навигация

5.2.2 Функциональные клавиши

Клавиша	Команда	Описание
F1	Справка	Показать/скрыть оверлей справки
F2	Закладки	Показать менеджер закладок
F3	Просмотр	Просмотр файла во встроенном просмотрщике
F4	Редактирование	Открыть файл во внешнем редакторе
F5	Копировать	Копировать в другую панель
F6	Переместить	Переместить в другую панель
F7	Каталог	Создать новый каталог
F8	Удалить	Удалить выделенный элемент
F10	Выход	Выход из приложения

5.2.3 Клавиши отметки

Клавиша	Действие
Insert	Переключить отметку выделенного файла
Пробел	Переключить отметку и перейти вниз
Ctrl+A	Отметить все файлы
Ctrl+U	Снять все отметки

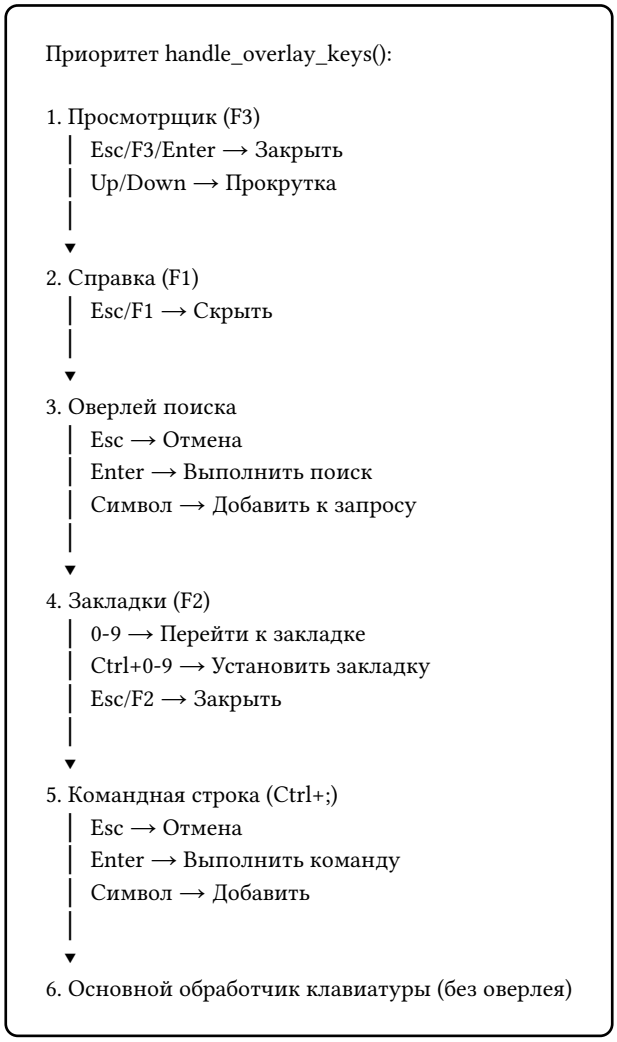
5.2.4 Быстрый поиск

Клавиша	Действие

Alt+буква	Добавить букву к поисковому запросу
Backspace	Удалить последний символ или очистить поиск
Esc	Очистить быстрый поиск

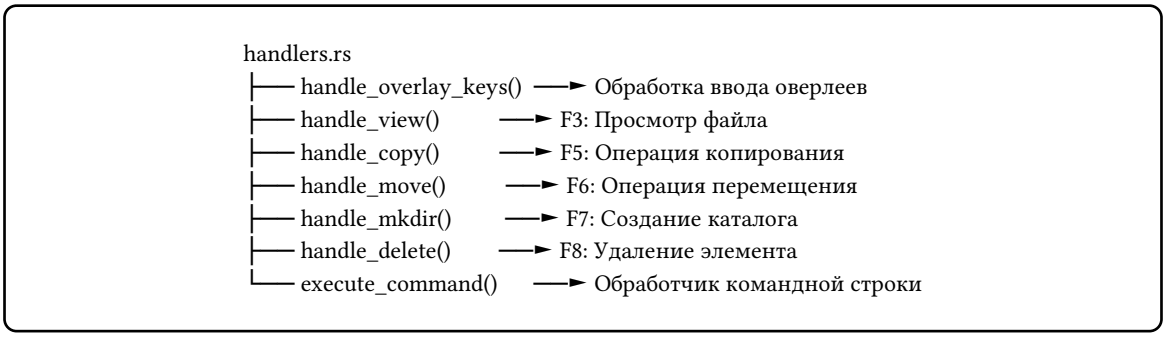
5.3 Приоритет оверлеев

Когда несколько оверлеев могут быть активны, они проверяются в следующем порядке:

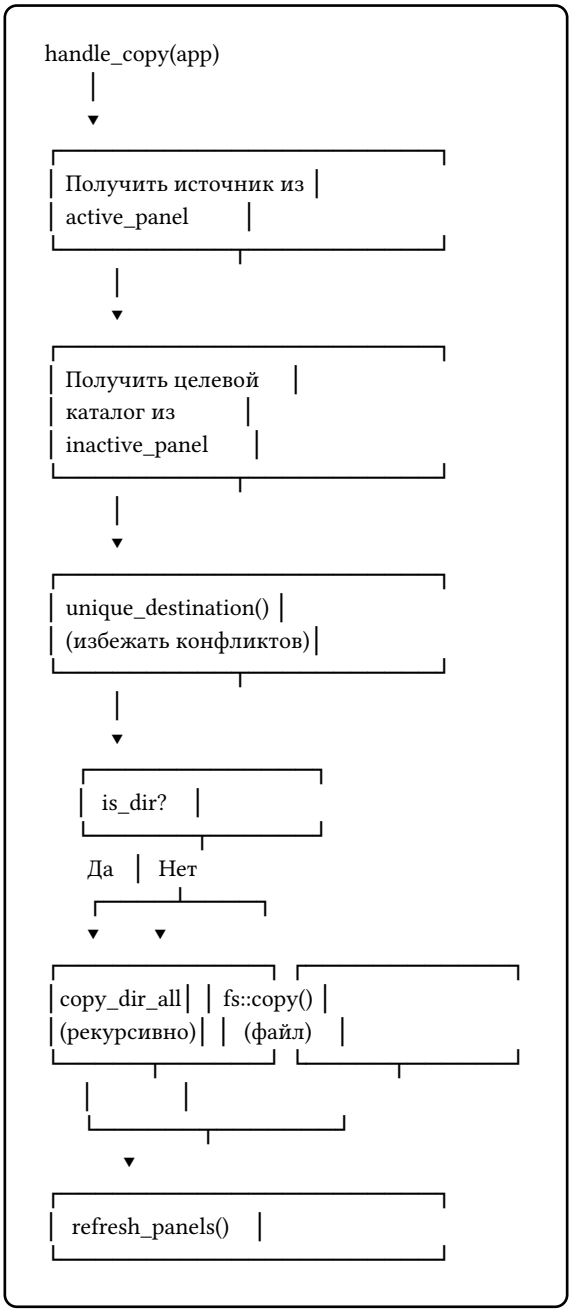


6 Обработчики файловых операций

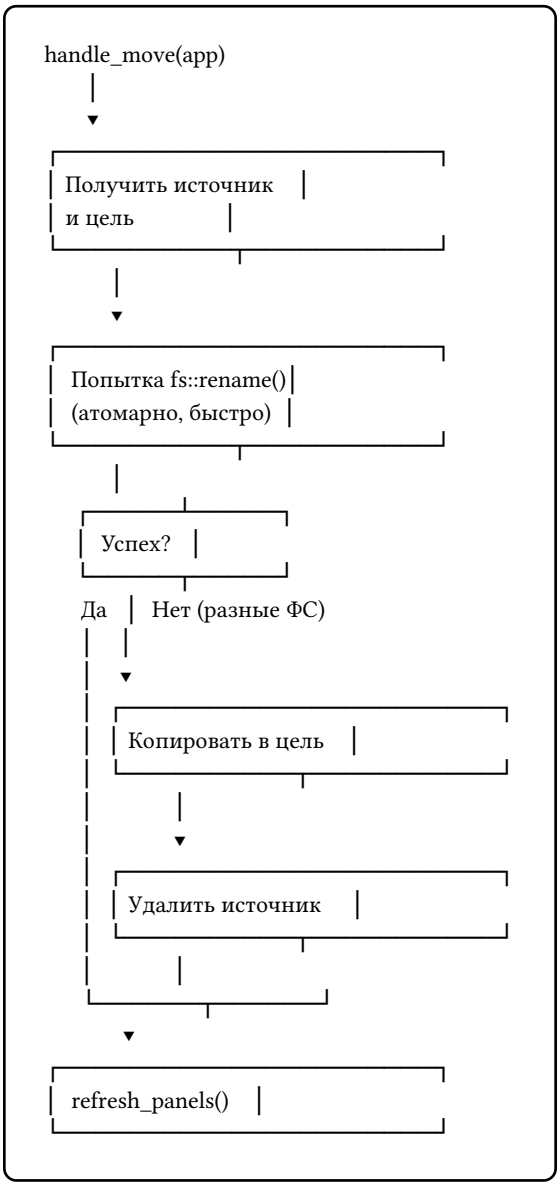
6.1 Обзор обработчиков



6.2 Поток операции копирования



6.3 Поток операции перемещения



6.4 Команды командной строки

Командная строка (активируется Ctrl+;) поддерживает следующие команды:

Команда	Аргументы	Действие
cd	<путь>	Сменить каталог (активная панель)
pwd		Показать текущий путь
sort	name|ext|size	Изменить режим сортировки
bookmark	set|go <0-9>	Управление закладками
mark	all	Отметить все файлы
unmark		Снять все отметки
help		Показать оверлей справки

quit		Выход из приложения

7 Вспомогательные функции

7.1 Обзор utils.rs

Функция	Назначение
format_size(bytes)	Преобразование байтов в человекочитаемый формат
unique_destination(base, name)	Генерация уникального имени файла
copy_dir_all(src, dst)	Рекурсивное копирование каталога
refresh_panels(app)	Перезагрузка обеих панелей

7.2 Логика format_size()

```
format_size(bytes: u64) → String

bytes < 1024           →  "{bytes} B"
bytes < 1 048 576      →  "{bytes/1024:.2} KB"
bytes < 1 073 741 824  →  "{bytes/1048576:.2} MB"
bytes >= 1 073 741 824 →  "{bytes/1073741824:.2} GB"

Примеры:
  512        →  "512 B"
  2048       →  "2.00 KB"
  5242880    →  "5.00 MB"
  2147483648 →  "2.00 GB"
```

7.3 Алгоритм unique_destination()

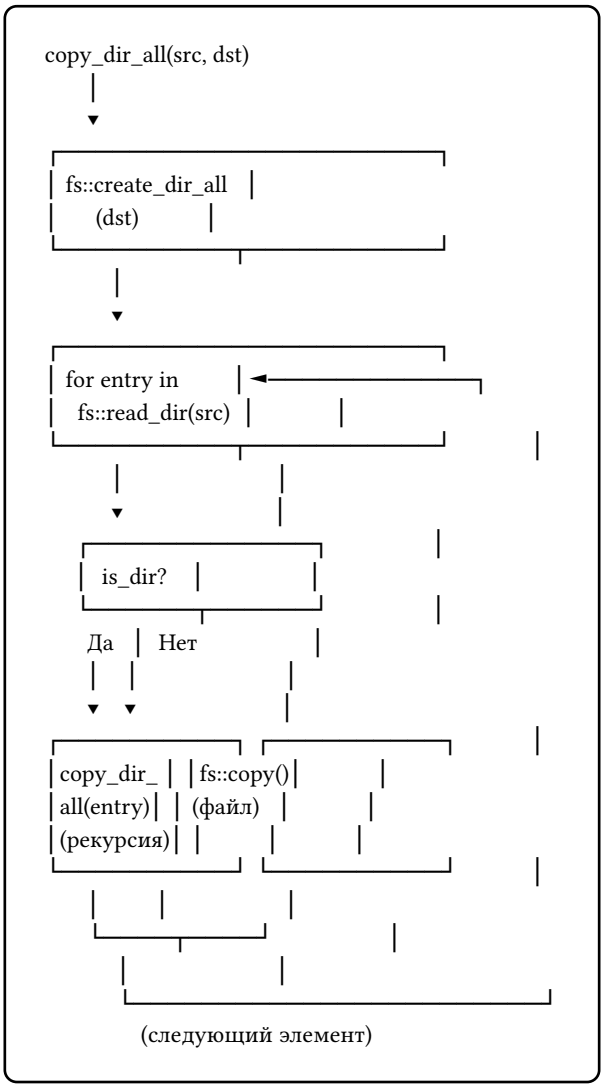
```
unique_destination(base: &Path, file_name: &OsStr) → PathBuf

Вход: base=/target, file_name="test.txt"

Проверка: /target/test.txt существует?
  Нет → вернуть /target/test.txt
  Да  → попробовать /target/test_1.txt
        → попробовать /target/test_2.txt
        → попробовать /target/test_3.txt
        → ... пока не найден уникальный

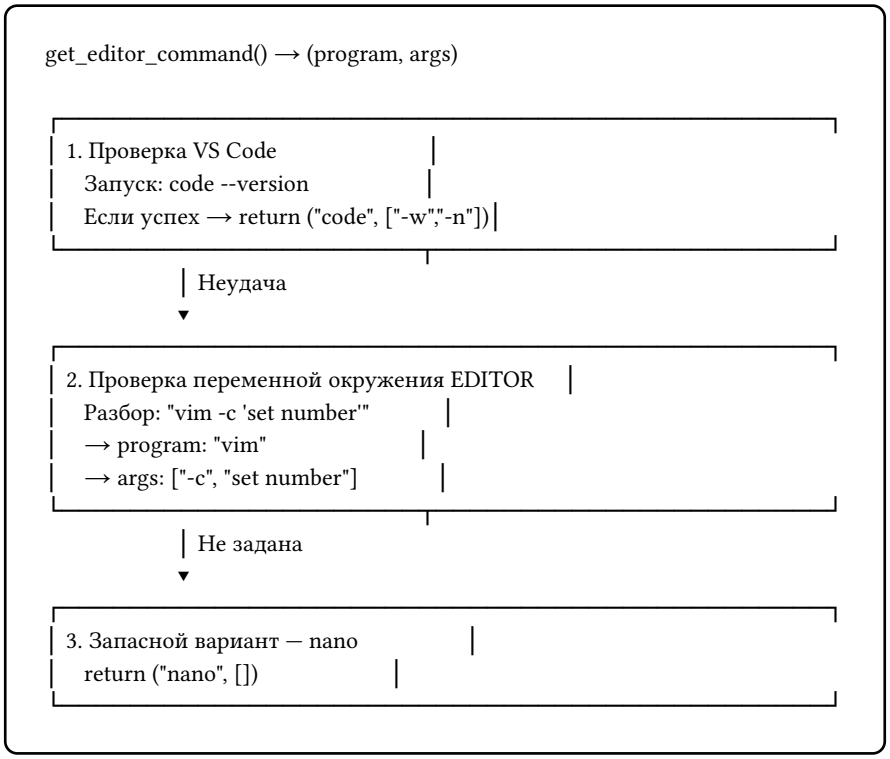
Особые случаи:
  • Dotfiles: ".env" → ".env_1"
  • Без расширения: "Makefile" → "Makefile_1"
```

7.4 Рекурсивный алгоритм copy_dir_all()

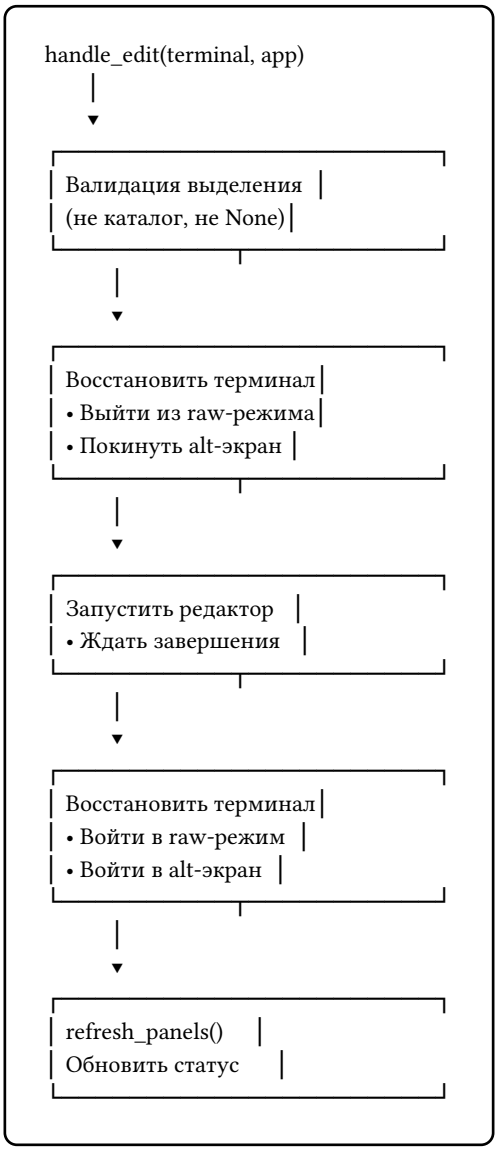


8 Интеграция с редактором

8.1 Стратегия выбора редактора

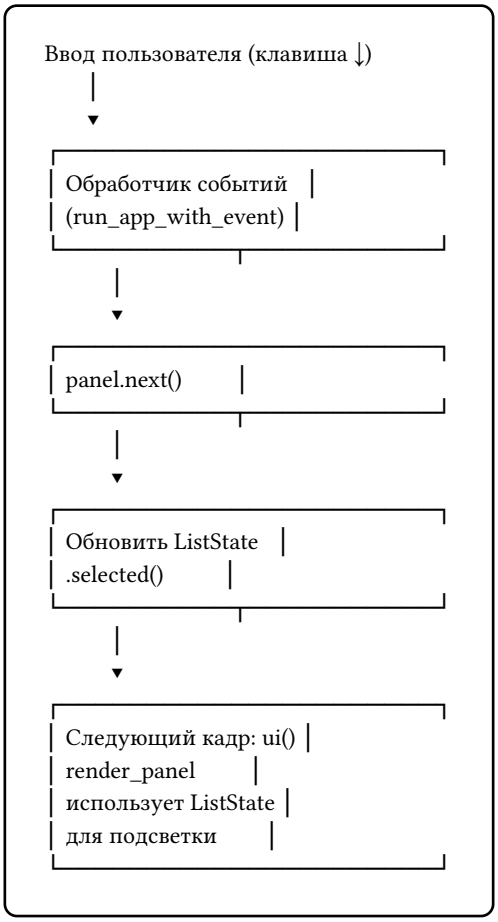


8.2 Последовательность handle_edit()

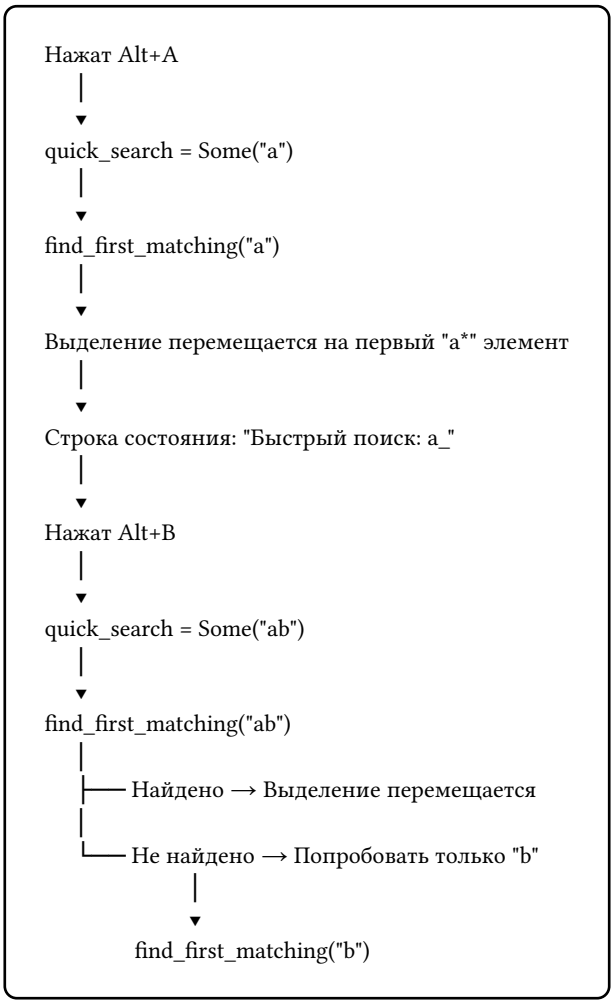


9 Паттерны потока данных

9.1 Поток навигации по панели



9.2 Поток быстрого поиска

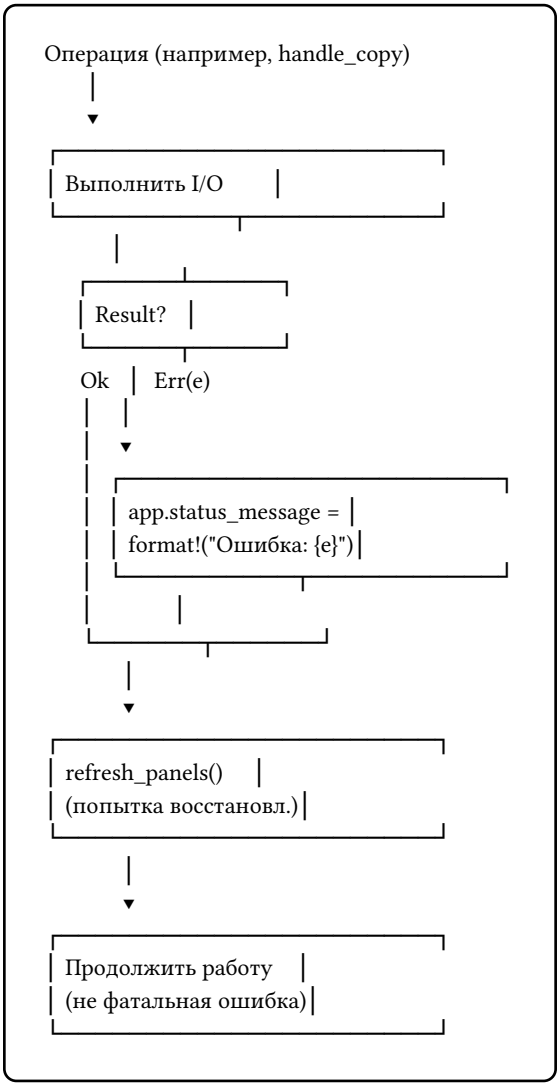


10 Обработка ошибок

10.1 Стратегия обработки ошибок

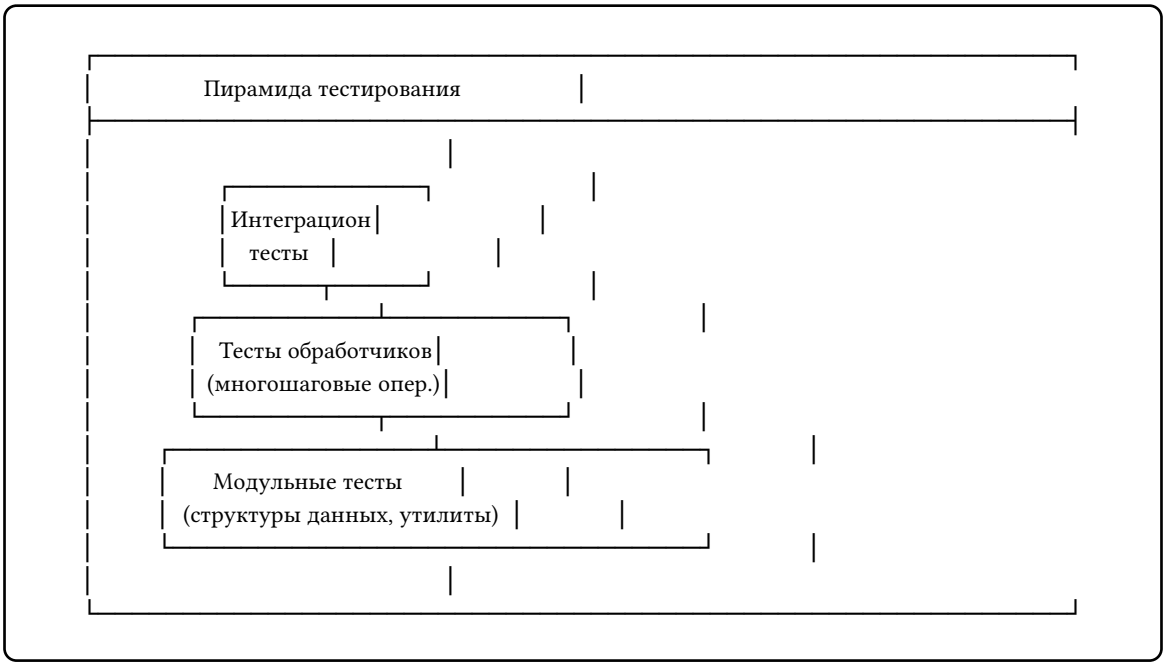
Операция	Тип ошибки	Обработка
Доступ к каталогу	io::Error	Показать в статусе, продолжить
Просмотр файла (не UTF8)	io::Error	Показать сообщение об ошибке
Копирование/Перемещение/Удаление	io::Error	Сообщение в статусе, обновить панели
Запуск редактора	Ошибка процесса	Статус: «Не удалось запустить»
Перезагрузка панели	io::Error	Попытка обновления в любом случае

10.2 Поток обработки ошибок



11 Архитектура тестирования

11.1 Уровни тестирования



11.2 Инфраструктура тестирования

Компонент	Назначение
WritableTestBackend	Пользовательский бэкенд ratatui для тестирования
tempfile::TempDir	Временные каталоги для тестов ФС
FORCE_RENAME_FAILURE	Имитация перемещения между устройствами
FORCE_COPY_DIR_ALL_STEP	Внедрение ошибок на определённых шагах
FORCE_REFRESH_ERROR	Тестирование ошибок обновления панелей

11.3 Точки мутационного тестирования

Кодовая база включает специальные точки внедрения для мутационного тестирования:

```
// В copy_dir_all()
if std::env::var("FORCE_COPY_DIR_ALL_STEP").is_ok() {
    return Err(io::Error::new(...));
}

// В handle_move()
if std::env::var("FORCE_RENAME_FAILURE").is_ok() {
    // Принудительный переход к копированию+удалению
}
```

12 Архитектурные решения

12.1 Ключевые проектные решения

12.1.1 1. Двухпанельная архитектура

Обоснование: Обеспечивает интуитивные операции источник-приёмник без диалогов.

Преимущества:
- Визуальное сравнение каталогов
- Файловые операции одной клавишей (F5, F6)
- Независимое состояние навигации для каждой панели

12.1.2 2. Паттерн стека оверлеев

Обоснование: Модальные взаимодействия без сложного управления окнами.

Преимущества:
- Простой конечный автомат (один оверлей за раз)
- Согласованное поведение Escape
- Лёгкое расширение новыми оверлеями

12.1.3 3. Сохранение выделения

Обоснование: Уменьшение дезориентации пользователя после операций.

Реализация:
- Сохранение имени выделенного файла перед перезагрузкой
- Поиск имени файла после перезагрузки
- Переход к первому элементу, если не найден

12.1.4 4. Переименование с запасным вариантом для перемещения

Обоснование: Корректная обработка перемещений между файловыми системами.

Стратегия:
1. Сначала попытка атомарного fs::rename() (быстрый путь)
2. При неудаче — копирование, затем удаление (медленный путь)
3. Пользователь видит одинаковый результат независимо от файловых систем

12.1.5 5. Уникальное именование файлов назначения

Обоснование: Никогда не перезаписывать существующие файлы молча.

Стратегия:
- Проверка существования файла назначения
- Добавление _1, _2 и т.д. к основе имени
- Сохранение оригинального расширения

13 Руководство по расширению

13.1 Добавление новых возможностей

13.1.1 Новая команда F-клавиши

```
// В lib.rs, блок match key.code:
KeyCode::F(9) => {
    handle_new_feature(&mut app)?;
}
```

13.1.2 Новый оверлей

```
// В models.rs, добавить в App:
pub new_overlay: Option<NewOverlayData>,

// В handlers.rs, handle_overlay_keys():
if let Some(ref mut overlay) = app.new_overlay {
    // Обработка клавиш специфичных для оверлея
    return true;
}

// В ui.rs, render_overlays():
if app.new_overlay.is_some() {
    render_new_overlay(frame, app);
}
```

13.1.3 Новый режим сортировки

```
// В models.rs, перечисление SortMode:
pub enum SortMode {
    Name,
    Extension,
    Size,
    Date,  // Новый режим
}

// В Panel::load_entries() добавить логику сортировки
```

13.1.4 Новая команда

```
// В handlers.rs, execute_command():
"newcmd" => {
    // Реализация команды
    app.status_message = "Готово".to_string();
}
```

13.2 Точки конфигурации

Настройка	Расположение	Текущее значение
Начальный каталог	lib.rs	env::current_dir()
Цветовая схема	ui.rs	Far Manager blue
Предпочтение редактора	editor.rs	VS Code > EDITOR > nano
Слоты закладок	models.rs	10 слотов (0-9)
Время антидребезга	lib.rs	250мс (режим покрытия)

14 Заключение

FunnyFileManager демонстрирует хорошо спроектированное терминальное приложение с:

14.1 Архитектурные особенности

Разделение ответственности	Отдельные модули для моделей, UI, обработчиков, утилит
Иммутабельная отрисовка	Функции UI не изменяют состояние
Комплексная обработка ошибок	Не фатальные ошибки, плавное восстановление
UX Far Manager	Знакомые горячие клавиши и цветовая схема
Тестируемый дизайн	Точки внедрения для мутационного тестирования
Безопасность терминала	Корректное восстановление режима для внешних программ

FunnyFileManager

Современный терминальный файловый менеджер на Rust

Построен с использованием Ratatui + Crossterm